

Single VM Appliance And Terraform Bridge

Goal

Run the entire Vyomi platform inside one host-launched Multipass VM and treat that VM as the durable appliance boundary.

The host OS should only:

- launch the VM,
- expose access to it,
- and restart it when needed.

Everything else should live inside the VM:

- simulator UI/API,
- CloudSim,
- provider services,
- VM-local runtime bridge,
- sandbox runtimes,
- and durable state storage.

The second half of the goal is to make the local simulator the source of truth for a Terraform export path so a user can move from local simulation to production deployment with a generated Terraform artifact.

Architecture

Host OS

- Starts one Multipass VM.
- Mounts or exposes the platform data volume.
- Does not run Vyomi business logic directly.

Appliance VM

- Runs the simulator API/UI.
- Runs the CloudSim backbone.
- Runs provider service emulators.
- Runs the VM-local runtime bridge.
- Stores simulator, CloudSim, and provider state on disk.
- Runs EC2-like sandboxes backed by LXD.
- Boots the platform from an appliance-specific compose file so the VM does not depend on host bridge or host-config wiring.

Canonical State

The platform keeps a canonical internal resource graph per active space. That graph is the source of truth for:

- the simulator UI,
- CloudSim summaries,
- provider operations,
- and Terraform export.

Terraform Bridge

The first implementation slice is a draft Terraform export API:

- it reads the active space,
- converts supported resources into Terraform JSON,
- emits matching Terraform HCL sidecar files from the same canonical graph,
- returns unsupported resources separately,
- and gives the user a structured export artifact.

This is intentionally not the final production deployer yet.

The intended progression is:

1. local simulation builds the resource graph,
2. the graph is exported to Terraform JSON,
3. Terraform validates the generated configuration,
4. Terraform plans/apply can deploy to real AWS/GCP/Azure accounts.
5. Terraform import can round-trip supported resources back into the simulator graph.

Current Implementation Slice

Implemented first:

- persisted CloudSim registry state,
- simulator state persistence,
- bridge-first host OS detection inside the appliance VM,
- and a Terraform export/plan/apply workflow for the active space.

The Terraform workflow currently produces a draft JSON artifact, matching HCL files, and a staged plan/apply bundle. It does not yet cover every provider resource shape or run a real Terraform CLI on every machine. It now also supports importing supported Terraform JSON back into the active simulator space so the graph can round-trip through Terraform export/import.

Next Steps

- Expand Terraform resource mappings per provider.
- Add download/save support in the UI.

- Add validation and plan generation.
- Add provider-specific modules for AWS, GCP, and Azure exports.